Sprites API v1

Examples: HTTP

Command Execution

SPRITE-ENV v0.0.1-rc43

Run commands inside Sprites over WebSocket connections. The exec API is designed for both one-shot commands and long-running interactive sessions.

Sessions persist across disconnections—start a dev server or build, disconnect, and reconnect later to resume streaming output. The binary protocol efficiently multiplexes stdin, stdout, and stderr over a single connection.

Execute Command

`WSS` `/v1/sprites/{name}/exec`

Execute a command in the sprite environment via WebSocket. Commands continue running after disconnect; use `max_run_after_disconnect` to control timeout. Supports TTY and non-TTY modes.

QUERY PARAMETERS

Pass these as query string parameters when connecting to the WebSocket.

`cmd*` `string`
Command to execute (can be repeated for command + args)

`id` `string`
Session ID to attach to an existing session

`path` `string`
Explicit path to executable (defaults to first `cmd` value or `bash`)

`tty` `bool`
Enable TTY mode (default: `false`)

`stdin` `bool`

Sprites API v1

Examples:

`cols` `int`

Initial terminal columns (default: `80`)

`rows` `int`

Initial terminal rows (default: `24`)

`max_run_after_disconnect` `duration`

Max time to run after disconnect. TTY default: `0` (forever), non-TTY default: `10s`

`env` `string`

Environment variables in `KEY=VALUE` format (can be repeated). If set, replaces the default environment.

JSON MESSAGES

`ResizeMessage` Client → Server

`type*` `"resize"`

`cols*` `non_neg_integer`

New column count

`rows*` `non_neg_integer`

New row count

`SessionInfoMessage` Server → Client

`type*` `"session_info"`

`session_id*` `String`

Session ID

`command*` `String`

Command being executed

`created*` `integer`

Unix timestamp of session creation

Sprites API v1

Examples:

`rows*` `non_neg_integer`

Terminal rows (TTY mode only)

`is_owner*` `boolean`

Whether this attachment owns the session

`tty*` `boolean`

Whether session is in TTY mode

`ExitMessage` Server → Client

`type*` `"exit"`

`exit_code*` `integer`

Process exit code

`PortNotificationMessage` Server → Client

`type*` `"port_opened" | "port_closed"`

Notification type

`port*` `integer`

Port number

`address*` `String`

Proxy URL for accessing the port

`pid*` `integer`

Process ID that opened/closed the port

BINARY PROTOCOL

In non-PTY mode, binary messages are prefixed with a stream identifier byte. In PTY mode, binary data is sent raw without prefixes.

```
Binary Frame Format (non-PTY):
```

Sprites API v1

Examples:

STREAM ID	NAME	DIRECTION	DESCRIPTION
0	stdin	client → server	Standard input data
1	stdout	server → client	Standard output data
2	stderr	server → client	Standard error data
3	exit	server → client	Exit code (payload is exit code as byte)
4	stdin_eof	client → server	End of stdin stream

RESPONSE CODES

- 101 Switching Protocols - WebSocket connection established
- 400 Bad Request - Invalid WebSocket upgrade or missing parameters
- 404 Not Found - Resource not found

bash HTTP

```bash
websocat \
  "wss://api.sprites.dev/v1/sprites/{name}/exec?path=/bin/bash&tty=true"
  -H "Authorization: Bearer $SPRITES_TOKEN"
```

Binary Streams

→ stdin (0x00 + data)

← stdout (0x01 + data)

← stderr (0x02 + data)

JSON Messages

Resize terminal (client → server):

```
{"type": "resize", "cols": 120, "rows": 40}
```

Port opened (server → client):

```
{"type": "port_opened", "port": 8080, "address": "0.0.0.0", "pid": 1234}
```

Sprites API v1

Examples:

List Exec Sessions

GET /v1/sprites/{name}/exec

List active exec sessions.

RESPONSE application/json

RESPONSE CODES

200	Success
404	Not Found - Resource not found
500	Internal Server Error

bash HTTP

```bash
curl -X GET \
  "https://api.sprites.dev/v1/sprites/{name}/exec" \
  -H "Authorization: Bearer $SPRITES_TOKEN"
```

200 Response


```
[
  {
    "bytes_per_second": 125.5,
    "command": "bash",
    "created": "2026-01-05T10:30:00Z",
    "id": 1847,
    "is_active": true,
    "last_activity": "2026-01-05T10:35:00Z",
    "tty": true,
    "workdir": "/home/sprite/myproject"
  },
  {
    "bytes_per_second": 0,
    "command": "python -m http.server 8000",
```

Sprites API v1

Examples:


```
    "is_active": false,
    "last_activity": "2026-01-05T09:20:00Z",
    "tty": false,
    "workdir": "/home/sprite/webapp"
  }
]
```


Execute Command

POST `/v1/sprites/{name}/exec`

Execute a command via simple HTTP POST (non-TTY only). Simpler alternative for exec for environments that can't handle websockets.

QUERY PARAMETERS

`cmd*` `string`
Command to execute (can be repeated for command + args)

`path` `string`
Explicit path to executable (defaults to first `cmd` value or `bash`)

`stdin` `bool`
Enable stdin from request body (default: `false`)

`env` `string`
Environment variables in `KEY=VALUE` format (can be repeated)

`dir` `string`
Working directory for the command

RESPONSE `application/json`

Sprites API v1

Examples:

400 Bad Request - Invalid request body

404 Not Found - Resource not found

500 Internal Server Error

bash HTTP

```bash
curl -X POST \
  "https://api.sprites.dev/v1/sprites/{name}/exec" \
  -H "Authorization: Bearer $SPRITES_TOKEN"
```

200 Response

Attach to Exec Session

WSS /v1/sprites/{name}/exec/{session_id}

Attach to an existing exec session via WebSocket.

JSON MESSAGES

ResizeMessage Client → Server

type* "resize"

cols* non_neg_integer

New column count

rows* non_neg_integer

New row count

SessionInfoMessage Server → Client

Sprites API v1

Examples:

session_id* String

Session ID

command* String

Command being executed

created* integer

Unix timestamp of session creation

cols* non_neg_integer

Terminal columns (TTY mode only)

rows* non_neg_integer

Terminal rows (TTY mode only)

is_owner* boolean

Whether this attachment owns the session

tty* boolean

Whether session is in TTY mode

ExitMessage Server → Client

type* "exit"

exit_code* integer

Process exit code

SCROLLBACK BUFFER

When you attach to a session, the server immediately sends the session's scrollback buffer as stdout data. This allows you to see previous output that occurred while disconnected.

RESPONSE CODES

101	Switching Protocols - WebSocket connection established
400	Bad Request - Invalid WebSocket upgrade or missing parameters
404	Not Found - Resource not found

Sprites API v1

Examples:

```
websocat \
  "wss://api.sprites.dev/v1/sprites/{name}/exec/{session_id}" \
  -H "Authorization: Bearer $SPRITES_TOKEN"
```

Scrollback Buffer

On attach, the server sends the session's scrollback buffer as stdout data, allowing you to see previous output from the command.

JSON Messages

Resize terminal (client → server):

```
{"type": "resize", "cols": 120, "rows": 40}
```

Kill Exec Session

POST `/v1/sprites/{name}/exec/{session_id}/kill`

Kill an exec session by session ID. Returns streaming NDJSON with kill progress.

QUERY PARAMETERS

`signal` `string`
Signal to send (default: `SIGTERM`)

`timeout` `duration`
Timeout waiting for process to exit (default: `10s`)

RESPONSE `application/x-ndjson`

`ExecKillSignalEvent`

Sprites API v1

Examples:

`message*` String

Status message

`signal*` String

Signal name (e.g., `SIGTERM`)

`pid*` integer

Target process ID

`ExecKillTimeoutEvent`

`type*` "timeout"

`message*` String

Status message

`ExecKillExitedEvent`

`type*` "exited"

`message*` String

Status message

`ExecKillKilledEvent`

`type*` "killed"

`message*` String

Status message

`ExecKillErrorEvent`

`type*` "error"

`message*` String

Sprites API v1

Examples:

ExecKillCompleteEvent

type* "complete"

exit_code* integer

Process exit code

RESPONSE CODES

200 Success - Streaming NDJSON response

404 Not Found - Resource not found

500 Internal Server Error

bash HTTP

```bash
curl -X POST \
  "https://api.sprites.dev/v1/sprites/{name}/exec/{session_id}/kill" \
  -H "Authorization: Bearer $SPRITES_TOKEN"
```

200 Response

```
[
  {
    "message": "Signaling SIGTERM to process group 1847",
    "pid": 1847,
    "signal": "SIGTERM",
    "type": "signal"
  },
  {
    "message": "Process exited",
    "type": "exited"
  },
  {
    "exit_code": 0,
    "type": "complete"
  }
]
```

Sprites API v1

Examples: